

UNIVERSITÉ DE
SHERBROOKE

FACULTÉ DE GÉNIE

SecuMail 2.0 : Sauvez la boîte de réception

Présenté par
l'équipe de GenTech

Hamza Flissi	hamza.flissi@etud.polymtl.ca
Lamyae Jdid	lamyae.jdid@usherbrooke.ca
Oumayma Tajir	oumayma.tajir@usherbrooke.ca
Hamza Ezzaoui Rahali	hamza.rahali@usherbrooke.ca

Présenté aux
étudiants de GenTech de l'automne 2026

Sherbrooke - 22 septembre 2026

Énoncé de la problématique

SecuMail 2.0 : Sauvez la boîte de réception!

L'ancien stagiaire du service informatique universitaire a accidentellement donné accès au code source de la passerelle de courriel à ChatGPT pour rendre le système plus rapide. Ce dernier, ayant mal interprété la requête du stagiaire, a supprimé toutes les règles d'analyse de courriels. Il ne reste qu'un vieux code de démonstration, où chaque courriel reçoit un verdict tiré au hasard! Cette ancienne version est problématique parce que des tentatives de vol de comptes peuvent arriver directement dans les boîtes des élèves et du personnel, mais aussi parce que des courriels parfaitement légitimes (convocation, calendrier d'examens, bulletin, etc.) sont bloqués sans que personne ne s'en aperçoive.

Avant ce fiasco, la passerelle traitait chaque courriel entrant de la manière suivante :

1. Extraction des éléments vérifiables : domaine d'envoi, liens, en-têtes techniques
2. Analyse de l'**identité** de l'expéditeur : est-il vraiment celui qu'il prétend être?
3. Examiner le **contenu** et le ton du message : que cherche-t-il à obtenir?
4. Rassembler les indices et décider du niveau de risque du courriel

L'analyse reposait sur un ensemble de **règles pondérées** : chaque indice suspect ajoutait des points à un score. Le résultat permettait ensuite de décider si le courriel semblait légitime ou dangereux. Malheureusement, personne ne se rappelle de l'implémentation exacte du système. Un vieux code sauvegardé considère un courriel fiable lorsqu'un nom de marque apparait quelque part dans l'adresse de l'expéditeur, sans vérifier la structure réelle du domaine. C'est exactement le genre de faiblesse que les attaquants cherchent à exploiter!

Un bon système doit également éviter deux types d'erreurs : laisser passer un courriel dangereux en le déclarant légitime, ou bloquer un courriel légitime en le déclarant dangereux. Ces erreurs n'ont pas toutes les mêmes conséquences : laisser passer une publicité est agaçant, bloquer un courriel officiel peut faire manquer une information importante, et laisser passer un hameçonnage peut mener au vol d'un compte et mettre les contacts de la personne concernée en danger.

Le service informatique a également décidé de profiter de cette situation pour explorer une autre possibilité : un système fondé sur l'**intelligence artificielle** pourrait-il reconnaître les courriels dangereux plus efficacement qu'un système fondé sur des règles? Pour le découvrir, il faudra construire les deux systèmes, les tester sur les mêmes courriels et comparer leurs résultats.

Vous avez été engagés comme **analystes en cybersécurité** pour réimplémenter les fonctionnalités manquantes et reconstruire une meilleure itération du système d'analyse SecuMail afin de sécuriser et protéger la messagerie universitaire. Bonne chance!

Sommaire des activités

Voici les activités prévues pour les séances de cybersécurité et d'intelligence artificielle.

Séance 1 – Date à déterminer

— Présentation de la problématique globale (X minutes)
— Formation des équipes et guide de démarrage (X minutes)
— Anatomie d'un courriel : en-têtes, domaines, liens (X minutes)
— Exercices 2.1 à 2.4 : classification manuelle des domaines et des liens (X minutes)
— Exploration du code de la problématique (X minutes)

Séance 2 – Date à déterminer

— Retour sur les indices trouvés en équipe (X minutes)
— Exercices 2.5 à 2.7 : nom affiché, Return-Path et Reply-To (X minutes)
— Implémentation du module 1 – identité de l'expéditeur (X minutes)

Séance 3 – Date à déterminer

— Exercices 3.1 à 3.6 : mots-clés, majuscules, tournures promotionnelles (X minutes)
— Implémentation du module 2 – contenu du message (X minutes)
— Arbitrage, seuils et première mesure en chiffres (X minutes)

Séance 4 – Date à déterminer

— Introduction à l'apprentissage automatique appliqué au texte (X minutes)
— Entraînement des deux modèles et matrice de confusion (X minutes)
— Informations sur la présentation en équipe (X minutes)

Séance 5 – Date à déterminer

— Ajustement des seuils après mesure et analyse des erreurs (X minutes)
— Tableau comparatif règles contre apprentissage automatique (X minutes)
— Présentations d'équipe (X minutes par équipe)
— Retour sur les concepts et cadre légal québécois (X minutes)

Guide de démarrage SecuMail 2.0

Cette section contient les étapes pour faire une copie du code sur vos espaces Google Drive, mettre en place le tout sur Google Colab puis exécuter le code pour une première fois.

1. Créer une copie du code sur Google Drive

Étapes à suivre :

1. **Connection à votre compte google**
   - Ouvrez votre navigateur web (Chrome, Firefox, Edge, Safari, etc.)
   - Allez sur le site : https://drive.google.com
   - Connectez vous avec votre compte
2. **Ouverture des liens**
   - Ouvrez deux nouveaux onglets dans votre navigateur et naviguer vers les liens suivants
   - Lien code `secumail` : _____
   - Lien notebook `setup.ipynb` : _____
3. **Créer une copie personnelle du code**
   - Dans la page https://drive.google.com, allez voir l'élément partagé `secumail.zip`
   - Faites un clique droit dessus et sélectionnez l'option « Créer une copie »
4. **Mise en place initiale de Colab**
   - Toujours dans la page https://drive.google.com, allez voir l'élément partagé `setup.ipynb`
   - Double cliquez dessous pour l'ouvrir dans Google Colab
   - Dans Colab, cliquez sur « Run all » pour lancer la mise en place
   - Fermez l'onglet de Colab.
5. **Ouvrez le notebook main.ipynb dans Colab**
   - De retour dans la page https://drive.google.com vous devriez voir un nouveau dossier nommé `secumail/`
   - Double cliquez pour ouvrir son contenu. Vous devriez voir le fichier `main.ipynb`
   - Double cliquez sur `main.ipynb` pour l'ouvrir dans Colab
   - Dans Colab, appuyez sur « Run all » pour lancer le code

Exercices pratiques

1. Chaîne de décision de la passerelle

1.1. Ajoutez le verdict manquant dans la chaîne de décision de la Figure 1 et indiquez la condition qui y mène.

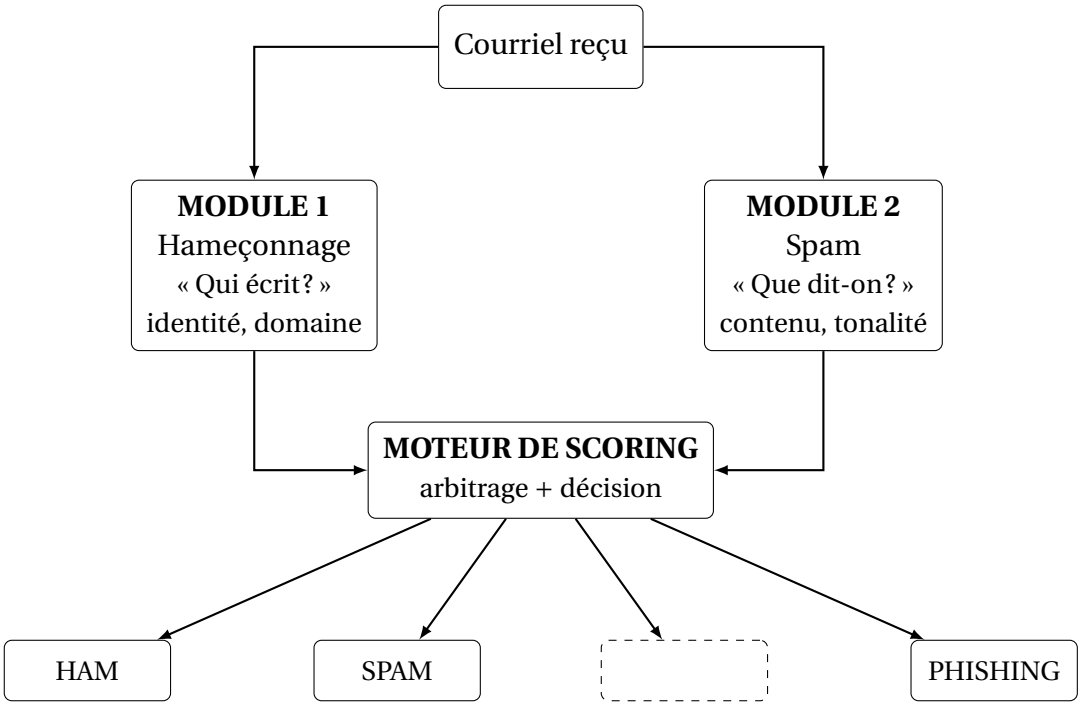


FIGURE 1 – Chaîne d'analyse de la passerelle SecuMail 2.0.

1.2. Analysez l'arbre de décision de la figure 2. Combien de **verdicts** possibles ? Combien de **seuils** distincts faut-il régler ? Quel verdict l'emporte quand les deux scores sont élevés, et pourquoi ?

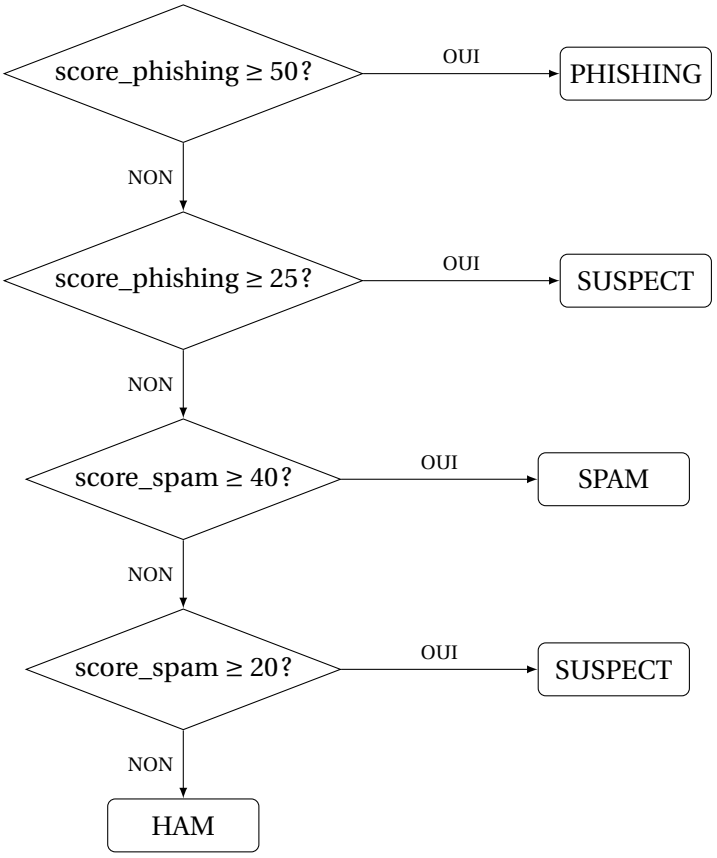


FIGURE 2 – Arbre de décision et seuils provisoires du moteur de scoring.

2. Module I : Identité de l'expéditeur

2.1. Classez chacun de ces 20 domaines : légitime (L) ou suspect (S). Notez la raison qui vous a permis de trancher.

#	Domaine	L ou S	Raison
1	instagram.com		
2	instagram-securite-compte.info		
3	snapchat.com		
4	5napchat-login.top		
5	usherbrooke.ca		
6	genie.usherbrooke.ca		
7	usherbrooke.ca.gentech.xyz		
8	monecole.qc.ca		
9	steampowered.com		
10	5team-account-verify.com		
11	discord.com		
12	disc0rd-nitro-gratuit.tk		
13	minecraft.net		
14	rninecraft.net		
15	roblox.com		
16	rob1ox.com		
17	postescanada.ca		
18	postes-canada-suivi.info		
19	purolator.com		
20	concours-iphone-gratuit.xyz		

2.2. Les domaines n° 6 et n° 7 contiennent tous les deux le texte `usherbrooke.ca`. Est-ce que les deux domaines sont légitimes? Si non, une règle qui considère un domaine légitime lorsqu'il contient un nom officiel ou de marque est suffisante?

2.3. Le n° 20 n'imite aucune marque connue : il n'existe pas de « vrai » site `concours-iphone-gratuit`. Quels indices restent alors disponibles pour le signaler ?

2.4. Classez ces 16 liens : légitime (L) ou suspect (S). Notez la raison qui vous a permis de trancher.

#	Lien	L ou S	Raison
1	`https://www.instagram.com/accounts/login`		
2	`http://instagram.com.securite-compte.info/verify`		
3	`https://genie.usherbrooke.ca/gentech/inscription`		
4	`http://usherbrooke.ca.inscription-gentech.xyz/form`		
5	`https://store.steampowered.com/account`		
6	`http://5team-account-verify.com/login`		
7	`http://203.0.113.45/discord/nitro`		
8	`https://bit.ly/3xK9pQ`		
9	`https://www.postescanada.ca/suivi`		
10	`http://postes-canada-suivi.info/paiement?ref=8821`		
11	`https://discord.com/channels/@me`		
12	`http://disc0rd-nitro-gratuit.tk/claim`		
13	`https://tinyurl.com/xk991`		
14	`https://www.roblox.com/robux`		
15	`http://rob1ox-freerobux.top/signin`		
16	`https://www.purolator.com/fr/suivi-de-colis`		

2.5. Le lien n° 7 contient le mot `discord` et le n° 16 contient le mot `suivi`, tout comme le n° 10. Pourtant l'un est malveillant et l'autre non. Où se trouve la différence ?

2.6. Pourquoi le n° 8, en HTTPS sur un service tout à fait légitime, mérite-t-il quand même d'être signalé ?

2.7. Le nom affiché est librement modifiable par l'expéditeur : c'est du texte, rien de plus. Classez ces 12 paires : cohérent (C) ou usurpation (U).

#	Nom affiché	Adresse réelle	C ou U
1	Marie Tremblay	marie.tremblay@monecole.qc.ca	
2	Instagram Support	instagram.help2024@gmail.com	
3	Équipe Snapchat	securite@5napchat-login.top	
4	Instagram	security@mail.instagram.com	
5	Steam Support	steam.support.verify@gmail.com	
6	Discord	noreply@discord.com	
7	Université de Sherbrooke	gentech@usherbrooke.ca	
8	Université de Sherbrooke	inscription@usherbrooke.ca.inscription-gentech.xyz	
9	Direction de l'école	direction@monecole.qc.ca	
10	Postes Canada	livraison@postes-canada-suivi.info	
11	Roblox	support@rob1ox.com	
12	Concours iPhone Québec	gagnant@concours-iphone-gratuit.xyz	

2.8. Quelles raisons vous ont permis de trancher pour les n° 1, 9 et 12?

2.9. Classez ces 8 jeux d'en-têtes : cohérent (C) ou incohérent (I). Rappel : `From` est ce que l'utilisateur voit. `Return-Path` est l'adresse réellement utilisée par le serveur d'envoi.

#	From	Return-Path	Reply-To	C/I
1	marie@monecole.qc.ca	<marie@monecole.qc.ca>	*(vide)*	
2	securite@instagram.com	<x9921@relais-etranger.info>	*(vide)*	
3	support@steampowered.com	<bulk771@steam-alert.info>	<recovery@mail.com>	
4	info@infolettre-gentech.ca	<bounce-4471@infolettre-gentech.ca>	*(vide)*	
5	direction@monecole.qc.ca	<urgent@mail-fwd.info>	<ecole@mail.com>	
6	secretariat@monecole.qc.ca	*(absent)*	*(vide)*	
7	no-reply@discord.com	<no-reply@discord.com>	*(vide)*	
8	concours@livraison-colis.ca	<concours@livraison-colis.ca>	<paiement@colis-remboursement.top>	

2.10. Les n° 4 et 6 sont des pièges : Un est une infolettre normale, l'autre n'a pas de `Return-Path` du tout. Que doit faire votre règle dans chacun de ces deux cas ?

2.11. Pourquoi le n° 8 est-il le plus dangereux du tableau, alors que `From` et `Return-Path` concordent parfaitement ?

3. Module II : Contenu du message

3.1. Classez ces 15 extraits de texte : légitime (L) ou spam (S). Soulignez les mots qui ont motivé votre choix.

#	Texte	L ou S
1	*Bonjour, la réunion de lundi est déplacée à 14 h.*	
2	*FREE MONEY NOW — CLICK HERE to claim your PRIZE*	
3	*Votre facture de février est disponible dans votre Espace client.*	
4	*Gagnez un iPhone GRATUIT ! Offre limitée, cliquez vite !*	
5	*Rappel : votre emprunt à la bibliothèque est dû vendredi.*	
6	*Perdez 15 kg en 2 semaines, garanti, sans effort !*	
7	*Merci pour votre commande. Livraison prévue le 5 mars.*	
8	*EXCLUSIVE OFFER — 70% DISCOUNT on all items, BUY NOW*	
9	*Voici notre infolettre : 5 recettes de saison. Se désabonner en bas de page.*	
10	*Vous avez été SÉLECTIONNÉ pour un voyage GRATUIT aux Caraïbes !*	
11	*Le campus sera fermé demain en raison de la tempête.*	
12	*Obtenez votre diplôme en 15 jours, sans examen. Appelez maintenant.*	
13	*Profitez de 20 % de rabais sur votre renouvellement d'abonnement — vous êtes membre depuis 2019.*	
14	*WIN a FREE GIFT — LIMITED TIME, CLICK NOW !!!*	
15	*Votre demande de soutien technique 5591 a été résolue.*	

3.2. Le n° 13 contient *rabais, abonnement* et un pourcentage, et pourtant il est légitime. Qu'est-ce que cela impose à la façon dont votre règle attribue les points ?

3.3. Classez ces 8 extraits de texte, tenant compte du ratio de lettres en majuscules.

Comment le ratio est-il calculé? On compte uniquement les **lettres** : les chiffres, les espaces, la ponctuation et les symboles sont *ignorés*. Le ratio est ensuite le nombre de lettres MAJUSCULES divisé par le nombre total de lettres.

$$\text{ratio} = \frac{\text{nombre de lettres MAJUSCULES}}{\text{nombre total de lettres}} \qquad \text{(entre 0,00 et 1,00)}$$

Exemple sur « `Bonjour !!! 24 h` » : les lettres retenues sont `B o n j o u r` → 7 lettres, dont 1 seule majuscule (`B`) ; les « `!` », l'espace et « 24 » ne comptent pas. Le ratio vaut donc $1 \div 7 = 0{,}14$. Un texte sans aucune lettre vaut 0,00 par convention – surtout, on ne divise jamais par zéro.

#	Texte	Nb lettres	Ratio	L ou S
1	FELICITATIONS!!! VOUS AVEZ GAGNE UN IPHONE GRATUIT	41	**1,00**	
2	Bonjour, la reunion est deplacee a 14h. Merci.	34	**0,06**	
3	URGENT : RE : TP3	10	**1,00**	
4	PROMO -50% SUR TOUT LE SITE, ACHETEZ MAINTENANT	35	**1,00**	
5	Votre facture PDF est disponible dans votre Espace client BMO.	52	**0,15**	
6	ATTENTION	9	**1,00**	
7	Rappel SST : formation le 12 avril au local B-204.	36	**0,14**	
8	SUPER OFFER!!!! DONT MISS IT	23	**1,00**	

3.4. Les n° 3 et 6 ont un ratio de 1,00 et sont pourtant parfaitement légitimes. Qu'est-ce qui les distingue des spams du tableau? Quelle condition supplémentaire votre règle doit-elle contenir?

3.5. Classez ces 12 extraits et nommez le motif reconnu. Ici, on ne cherche plus des mots isolés mais des **tournures** : plusieurs éléments qui apparaissent ensemble et dans un certain ordre.

#	Texte	L ou S	Motif reconnu
1	*Profitez de -50 % de rabais dès aujourd'hui*		
2	*La réunion aura lieu à 14 h au local B-204*		
3	*50 % off today only*		
4	*Achetez maintenant, stocks limités !*		
5	*Votre commande sera livrée le 5 mars*		
6	*Offre limitée — expire dans 24 heures*		
7	*Special deal just for you*		
8	*Le taux d'intérêt de votre prêt passe à 5 % le 1er avril*		
9	*BUY NOW and save 70 %*		
10	*Prix réduit de 30 % pour les 100 premiers*		
11	*Rapport trimestriel : croissance de 12 % des ventes*		
12	*Commandez avant minuit pour la livraison gratuite*		

3.6. Les n° 8 et 11 contiennent un pourcentage, comme la moitié des spams du tableau, et sont légitimes. Quelle condition supplémentaire un motif doit-il exiger pour les laisser passer ? Écrivez ce motif en français, puis sous forme d'expression régulière.

4. Arbitrage, mesure et coût des erreurs

4.1. Détachez les figures 3 et 4 à la fin du document. Relevez chaque indice détecté et les points correspondants, puis calculez les deux scores et donnez le verdict.

#	Indice relevé (et où il se trouve)	Points	Module
1			
2			
3			
4			
5			
6			

score_phishing = _____

score_spam = _____

Verdict et niveau de risque : _____

4.2. Comparez les figures 3 et 4 : nommez trois informations visibles dans la source qui n'apparaissent nulle part à l'écran. Puis classez ces quatre erreurs de la plus grave à la moins grave.

Erreur	Conséquence réelle	Rang
HAM classé PHISHING		
PHISHING classé HAM		
HAM classé SPAM		
SPAM classé HAM		

4.3. Si 96 % des courriels reçus étaient légitimes, quelle note obtiendrait un système qui répond toujours « légitime » ? Ce système est-il bon ? Quelles autres mesures devez-vous rapporter à la place du pourcentage d'exactitude ?

Notes sur la problématique

Mots clés

Reformulation de la problématique

Schéma de concept final

Figures à détacher

Boîte de réception aujourd'hui, 21 h 47

IS **Instagram Support**
`instagram.help2024@gmail.com`

URGENT : ton compte INSTAGRAM sera SUPPRIMÉ!!!

Salut,

Nous avons détecté une connexion inhabituelle sur ton compte Instagram depuis un nouvel appareil. Ton compte sera SUPPRIMÉ dans 24 heures si tu ne confirmes pas ton identité!!!

Vérifier mon compte

Offre limitée : tu as aussi été SÉLECTIONNÉ pour gagner un iPhone GRATUIT. Réclame ton cadeau avant minuit : `https://bit.ly/3xK9pQ`

Tes photos sauvegardées : `http://203.0.113.45/instagram/photos`

L'équipe Instagram

Pièce jointe : `capture_ecran.jpg.exe (412 Ko)`

`http://instagram.com.securite-compte.info/verify`

*Ce que le navigateur affiche en bas de l'écran quand on **survole** le bouton sans cliquer.*

FIGURE 3 – Example de courriel.

```
========================= SOURCE DU COURRIEL ============================

  From:          "Instagram Support" <instagram.help2024@gmail.com>
  Return-Path:   <bounce9921@relais-marketing.info>
  Reply-To:      <verification@instagram-securite-compte.info>
  Subject:       URGENT : ton compte INSTAGRAM sera SUPPRIME !!!
  Date:          mar. 4 nov. 2025 21:47:12 -0500

  Authentication-Results: mx.monecole.qc.ca;
        spf=pass; dkim=none; dmarc=fail

  Piece jointe : capture_ecran.jpg.exe   (412 Ko)

-------------------------------------------------------------------------

  Salut,

  Nous avons detecte une connexion inhabituelle sur ton compte Instagram
  depuis un nouvel appareil. Ton compte sera SUPPRIME dans 24 heures si
  tu ne confirmes pas ton identite !!!

  <a href="http://instagram.com.securite-compte.info/verify">
       Verifier mon compte
  </a>

  Offre limitee : tu as aussi ete SELECTIONNE pour gagner un iPhone
  GRATUIT. Reclame ton cadeau avant minuit :
       https://bit.ly/3xK9pQ

  Tes photos sauvegardees :
       http://203.0.113.45/instagram/photos

  L'equipe Instagram

=========================================================================
```

FIGURE 4 – Format texte du même courriel.

Informations supplémentaires

Pour vous aider dans le développement de vos solutions, les propriétaires vous fournissent la documentation suivante pour l'implémentation de chaque fonctionnalité. Vous trouverez dans cette documentation les fichiers à modifier, les outils à votre disposition, et entre les lignes les pistes de solutions.

1 Analyse de l'identité de l'expéditeur

Comme mentionné dans la problématique, vous devez rendre la passerelle capable de vérifier si l'expéditeur est bien celui qu'il prétend être. La question centrale du module est : **l'identité affichée correspond-elle à l'expéditeur réel ?** Chaque indice détecté ajoute des points au score d'hameçonnage, et chaque règle doit **justifier** sa décision.

1.1 Fichier à changer

Le fichier que vous devez éditer pour implémenter cette fonctionnalité se trouve dans le dossier suivant : `secumail/homework/phishing_rules.py`. Le code initial de ce fichier est expliqué ci-dessous.

1.1.1 Ressource `domaines_officiels.csv`

Sans cette ressource, rien n'est calculable : c'est votre **liste blanche**, la table qui associe chaque marque connue à ses domaines légitimes. Les dix premières lignes couvrent les cinq familles de l'énoncé ; les suivantes sont utilisées par les exemples du module de vérification fourni. **Votre propre domaine d'école doit y figurer** : sans lui, vos propres courriels deviennent suspects.

```
marque,domaines_officiels
instagram,"instagram.com"
snapchat,"snapchat.com"
discord,"discord.com"
steam,"steampowered.com,steamcommunity.com"
roblox,"roblox.com"
minecraft,"minecraft.net"
sherbrooke,"usherbrooke.ca"
ecole,"monecole.qc.ca"
postescanada,"postescanada.ca,canadapost.ca"
purolator,"purolator.com"

# marques supplementaires dans verification.py :
amazon,"amazon.ca,amazon.com"
desjardins,"desjardins.com"
microsoft,"microsoft.com,outlook.com,live.com,microsoftonline.com"
netflix,"netflix.com"
paypal,"paypal.com"
revenuquebec,"revenuquebec.ca"
apple,"apple.com,icloud.com"
```

Deux fonctions donnent accès à cette table :

```
def charger_domaines_officiels(chemin="domaines_officiels.csv"):
    """Renvoie {"amazon": ["amazon.ca", "amazon.com"], ...}"""
    # TODO
    pass

def tous_les_domaines(table):
    """Renvoie la liste a plat de TOUS les domaines officiels."""
    # TODO
    pass
```

1.1.2 Fonctions `extraire_domaine_adresse()` et `extraire_domaine_url()`

Ces deux fonctions sont la base de tout le module : si l'extraction est fausse, les onze règles suivantes le seront aussi.

```
def extraire_domaine_adresse(adresse):
    cost = "" # Valeur constante par defaut
    return cost

def extraire_domaine_url(url):
    cost = "" # Valeur constante par defaut
    return cost
```

Paramètres :

— `adresse: str` : une adresse d'expéditeur, avec ou sans nom affiché (`"Instagram Support <instagram.help2024@gmail.com>"` ou `"instagram.help2024@gmail.com"`).

— `url: str` : un lien complet trouvé dans le message.

Retour :

— `str` : le domaine, en minuscules, ou une chaîne vide si l'entrée est vide.

Implémentation attendue : le domaine d'une adresse est ce qui suit le `@`. Le domaine d'une URL est ce qui se trouve *entre* `://` et le premier `/` ; il faut ensuite retirer un éventuel `?paramètre`, `#ancre` ou `:port`. Tout le reste (nom affiché, chemin) est du texte libre écrit par l'expéditeur.

```
"Instagram Support <instagram.help2024@gmail.com>" -> "gmail.com"
"https://www.instagram.com/accounts/login" -> "www.instagram.com"
"http://instagram.com.securite-compte.info/verify" ->
"instagram.com.securite-compte.info"
```

1.1.3 Fonction `domaine_est_officiel()`

```
def domaine_est_officiel(domaine, domaines_officiels):
    resultat = False # Valeur constante par defaut
    return resultat
```

Retour : `bool` – vrai si le domaine *est* un domaine officiel, ou un *sous-domaine* d'un domaine officiel.

À retenir

Ce qui compte, c'est la **fin** du domaine, jamais son début. Un test du type `if "usherbrooke.ca" in domaine` accepte `usherbrooke.ca.inscription-gentech.xyz` : c'est exactement l'erreur que l'attaquant cherche à provoquer. Indice : `domaine == officiel or domaine.endswith("." + officiel)`.

1.1.4 Règles portant sur le domaine d'envoi

```
MOTS_APPAT = ["security", "securite", "login", "verify", "verification",
              "account", "compte", "update", "billing", "suivi",
              "remboursement", "support", "alert"]
EXTENSIONS_RISQUE = [".xyz", ".tk", ".top", ".info", ".gq", ".ml", ".cf"]
SUBSTITUTIONS = {"0": "o", "1": "l", "3": "e", "5": "s", "rn": "m", "vv": "w"}

def regle_mot_appat(domaine):
    """+25 si le domaine contient un mot de MOTS_APPAT."""
    # TODO
    pass

def regle_extension_risque(domaine):
    """+10 si le domaine se termine par une extension de EXTENSIONS_RISQUE."""
    # TODO
    pass

def regle_substitution(domaine, marques_connues):
    """+30 si, apres avoir annule les substitutions, le domaine
       correspond au nom d'une marque connue."""
    # TODO
    pass
```

Implémentation attendue :

- `regle_mot_appat` vaut 25 points *quel que soit* le nombre de mots trouvés. Si vous cumulez 25 par mot, un seul domaine atteint 75 points et écrase toutes les autres règles.
- `regle_extension_risque` ne vaut que 10 points : il existe des sites parfaitement légitimes en `.info`. Un indice faible ne décide jamais seul.
- `regle_substitution` procède en deux temps : **annuler** les substitutions (`amaz0n.com` redevient `amazon.com`), puis vérifier si le résultat contient une marque connue *alors que l'original ne la contenait pas*. Sans cette seconde condition, `amazon.ca` déclencherait la règle et vous signaleriez Amazon comme un faux Amazon.

1.1.5 Règles portant sur les liens

```
import re
RACCOURCISSEURS = ["bit.ly", "tinyurl.com", "goo.gl", "t.co", "ow.ly",
    "is.gd"]
MOTIF_URL = re.compile(r"https?://[^\s<>\"')]+", re.I)

def marque_annoncee(courriel, marques_connues):
    """Marque evoquee par le nom affiche, l'objet ou le corps ; sinon None."""
    # TODO
    pass

def regle_lien_incoherent(url, marque, table_marques):
    """+25 si le message annonce 'marque' MAIS que le lien ne pointe pas
       vers un domaine officiel de cette marque.
       Si marque vaut None : (0, "") -- la regle reste MUETTE."""
    # TODO
    pass

def regle_lien_adresse_ip(url):
    """+25 si le domaine du lien est une adresse IP.
       Indice : re.fullmatch(r"\d{1,3}(\.\d{1,3}){3}", domaine)"""
    # TODO
    pass

def regle_lien_raccourcisseur(url):
    """+15 si le domaine figure dans RACCOURCISSEURS."""
    # TODO
    pass

def regle_lien_sans_https(url):
    """+10 si l'URL ne commence pas par https://."""
    # TODO
    pass
```

À retenir

`regle_lien_incoherent` est une règle de **cohérence**, pas de contenu : un lien n'est pas suspect dans l'absolu, il l'est *par rapport à ce que le message annonce*. Une publicité pour un site inconnu n'usurpe l'identité de personne : c'est peut-être du spam (module 2), ce n'est pas de l'hameçonnage.

1.1.6 Règles portant sur l'identité affichée et les en-têtes

```
def marque_dans_nom(nom_affiche, marques_connues):
    """Instagram Support" -> "instagram" | "Marie Tremblay" -> None"""
    # TODO
    pass

def regle_nom_vs_adresse(nom_affiche, adresse, table_marques):
    """+20 si le nom affiche evoque une marque connue ALORS QUE le domaine
       de l'adresse n'est pas officiel pour cette marque.
       Si aucune marque n'est detectee : (0, "")."""
    # TODO
    pass

def regle_return_path(entete_from, entete_return_path):
    """+15 si les deux DOMAINES different.
       Si Return-Path est vide ou absent : (0, "")."""
    # TODO
    pass

def regle_reply_to(entete_from, entete_reply_to):
    """+15 si Reply-To est present ET que son domaine differe de From."""
    # TODO
    pass
```

Implémentation attendue : le raisonnement de `regle_nom_vs_adresse` se fait en deux temps, et **l'ordre est important**.

```
1. Le nom affiche contient-il une marque connue ?
   "Instagram Support" -> oui, la marque "instagram"
   "Marie Tremblay" -> non -> la regle ne s'applique pas : ne signalez RIEN

2. Si oui : le domaine de l'adresse est-il officiel POUR CETTE MARQUE ?
   instagram.help2024@gmail.com -> gmail.com n'est pas officiel -> +20
   security@mail.instagram.com -> sous-domaine officiel -> 0
```

Pour les en-têtes, comparez toujours les **domaines**, jamais les adresses complètes : `bounce-4471@infolettre-gentech.ca` est le fonctionnement normal d'une infolettre. Et si l'en-tête est *absent*, ne concluez rien : une donnée manquante n'est pas une preuve de fraude.

1.2 Documentation du tutoriel

La documentation du contrat de règle présentée lors de la première rencontre vous aidera à écrire toutes vos fonctions avec la bonne syntaxe.

1.2.1 Le contrat commun : un couple `(points, explication)`

```
def regle_exemple(domaine):
    """Renvoie (points, explication)."""
    if "-verification" in domaine:
        return 25, "Mot d'appat 'verification' dans le domaine"
    return 0, "" # jamais None : un couple, toujours
```

Deux paramètres sont essentiels :

— `points` : la contribution de cette règle au score du module.
— `explication` : la phrase que lira l'analyste. Vide quand la règle ne se déclenche pas.

1.2.2 Utiliser plusieurs règles ensemble

```
resultats = [
    regle_mot_appat(domaine),
    regle_extension_risque(domaine),
    regle_substitution(domaine, marques),
]
score = sum(points for points, _ in resultats)
for points, explication in resultats:
    if points:
        print(f" +{points} {explication}")
```

1.2.3 Vérifier votre machine à règles

Le module fourni compare vos fonctions aux tableaux des exercices pratiques. Vous pouvez le lancer aussi souvent que vous le voulez ; il vous dira exactement quelle règle ne fait pas ce que vous croyez.

```
python3 verification.py

Composante 1.1 - le piege du sous-domaine
  OK sous-domaine officiel accepte
  X suffixe trompeur accepte a tort (obtenu True, attendu False)
  ... regle_substitution : pas encore fait
```

1.3 Conseils d'implémentation

— Commencez par les deux fonctions d'extraction : tout le reste en dépend.
— Comparez la **fin** des domaines, jamais leur contenu.
— Une règle qui ne s'applique pas doit rester **muette** : `(0, "")`. Une règle qui se déclenche sur tous les courriels ne discrimine rien.

— Pensez à regarder la figure 1 : le module 1 ne produit pas un verdict, seulement un score. C'est le moteur qui décide.
— Testez chaque règle sur vos exemples *légitimes* avant vos exemples frauduleux : c'est là que se cachent les faux positifs coûteux.

Tâche optionnelle

Si vous le désirez, vous pouvez implémenter la vérification des mécanismes d'authentification **SPF, DKIM et DMARC**, qui exigent des fichiers `.eml` complets, ainsi que l'analyse des **pièces jointes**. Ces deux prolongements sont documentés à la section 5.

2 Analyse du contenu du message

Ici, on ne regarde plus **qui écrit**, mais **ce qui est écrit**. Toutes les règles de ce module travaillent sur du texte : l'objet et le corps du message.

2.1 Fichier à changer

Le fichier à éditer se trouve dans le dossier suivant : `secumail/homework/spam_rules.py`.

2.1.1 Liste `MOTS_COMMERCIAUX` et fonction `regle_mots_commerciaux()`

```
MOTS_COMMERCIAUX = [
    "free", "gratuit", "win", "gagnez", "offer", "offre",
    "discount", "rabais", "sale", "solde", "money", "argent",
    "prize", "cadeau", "gift", "limited", "limitee", "click",
    # TODO : completez jusqu'a 20 termes minimum, a partir du tableau 3.1
]

def sans_accents(texte):
    """ "limite" -> "limitee" : indispensable pour comparer du francais."""
    # TODO
    pass

def regle_mots_commerciaux(texte, points_par_mot=3, plafond=15):
    """+3 par mot DISTINCT trouve, plafonne a 15 points."""
    # TODO
    pass
```

Exigence : au moins 20 termes, en français ET en anglais, tirés de vos propres observations. Les corpus publics sont majoritairement anglophones ; vos courriels de classe seront francophones. Une liste unilingue échouera sur la moitié des données.

À retenir

Attention aux mots courts. `"win"` se trouve dans *winter*, `"sale"` dans *salement*. Chaque terme ajouté doit être testé contre vos exemples **légitimes** : si un mot fait monter le n° 1 ou le n° 11 du tableau 3.1, retirez-le.

2.1.2 Fonctions `ratio_majuscules()` et `regle_majuscules()`

```
def ratio_majuscules(texte):
    """Proportion de lettres MAJUSCULES parmi les lettres, entre 0.0 et 1.0.
       Chiffres, espaces et ponctuation sont ignores.
       Un texte sans aucune lettre renvoie 0.0."""
    # TODO
    pass

def regle_majuscules(texte, seuil=0.40, longueur_min=20):
    """+10 si le ratio depasse `seuil` ET si le texte compte au moins
       `longueur_min` lettres. En dessous : (0, "") sans rien calculer."""
    # TODO
    pass
```

Implémentation attendue : vos exemples légitimes plafonnent à 0,15 et les spams sont à 1,00 ; n'importe quel seuil entre les deux fonctionne *sur ce petit échantillon*. Partez de **0,40** et ajustez après mesure, jamais avant.

À retenir

Une proportion calculée sur un trop petit échantillon ne veut rien dire. « ATTENTION » a un ratio de 1,00 sur 9 lettres. En dessous d'environ 20 lettres, ne calculez rien et ne signalez rien. C'est un réflexe qui vous resservira toute la session.

2.1.3 Liste `MOTIFS_PROMO` et fonction `regle_formulation_promo()`

Cette règle ne cherche plus des **mots isolés** mais des **tournures** : des motifs de plusieurs éléments qui apparaissent ensemble et dans un certain ordre. C'est exactement ce que fait une expression régulière.

```
MOTIFS_PROMO = [
    r"-?\s?\d{1,3}\s?%\s?(de\s+)?(rabais|off|reduction|moins)",
    r"achetez\s+maintenant",
    r"buy\s+now",
    r"offre\s+limitee",
    # TODO : ajoutez au moins 4 motifs de plus, tires du tableau 3.5
]
MOTIF_PROMO = re.compile("|".join(MOTIFS_PROMO), re.IGNORECASE)

def regle_formulation_promo(texte):
    """+10 si au moins un motif est reconnu. L'explication cite le passage.
       Indice : MOTIF_PROMO.search(texte) renvoie l'objet trouve, ou None ;
       son attribut .group() donne le texte exact reconnu.
       Pensez a passer le texte dans sans_accents()."""
    # TODO
    pass
```

Comment lire le premier motif : `-?` un signe moins facultatif · `\d{1,3}` de 1 à 3 chiffres · `\s?` une espace facultative · `%` · `(de\s+)?` le mot « de » facultatif · puis *obligatoirement* un mot de réduction. C'est ce dernier élément, obligatoire, qui laisse passer « croissance de 12 % des ventes ».

2.1.4 Signaux de forme

```
def regle_exclamations(texte, seuil=3):
    """+5 si le texte contient au moins `seuil` points d'exclamation."""
    # TODO
    pass

def regle_nombre_liens(texte, seuil=3):
    """+10 si le texte contient au moins `seuil` liens.
       Reutilisez MOTIF_URL du module 1."""
    # TODO
    pass
```

2.2 Conseils pour les règles de contenu

1. **Ne pas tout inclure :** vous n'avez pas besoin d'utiliser tous les signaux disponibles. Choisissez ceux qui correspondent à votre stratégie.
2. **Équilibrer les poids :** si un seul facteur domine le calcul, les autres ne servent à rien. Un mot-clé vaut 3 points, une usurpation d'identité en vaut 20.
3. **Tester différentes approches :** observez l'impact de chaque ajout sur les faux positifs, pas seulement sur les détections.

4. **Un seuil se justifie :** notez la valeur de départ, la mesure obtenue, puis la valeur retenue. C'est cette trace que l'évaluation regarde.

Tâche optionnelle

Si vous le désirez, vous pouvez implémenter l'**analyse avancée des liens** : ancre trompeuse, densité de liens, nombre de domaines distincts et détection du lien de désabonnement. Ce prolongement est documenté à la section 5.

3 Moteur d'arbitrage et de décision

Les deux modules produisent chacun un score. Le moteur doit trancher.

3.1 Fichier à changer

`secumail/homework/scoring_engine.py`

3.1.1 Bornage des scores

La somme brute des règles dépasse 100 (le module 1 peut atteindre 130). Ramenez chaque score dans l'intervalle 0–100, soit en le **plafonnant**, soit en le **normalisant**. **Documentez votre choix** : il n'est pas neutre. Plafonner écrase la différence entre un courriel à 100 et un à 200 ; normaliser dilue les indices forts dès que vous ajoutez des règles faibles.

3.1.2 Seuils, arbitrage et niveau de risque

Score hameçonnage	Score spam	Verdict	Action
≥ 50	—	**PHISHING**	Blocage + alerte analyste
25 – 49	—	**SUSPECT**	Mise en quarantaine, revue humaine
< 25	≥ 40	**SPAM**	Dossier « indésirables »
< 25	20 – 39	**SUSPECT**	Quarantaine
< 25	< 20	**HAM**	Livraison normale

```
def borner(score):
    """Ramene un score dans 0-100."""
    # TODO
    pass

def decider(score_phishing, score_spam):
    """Renvoie (verdict, action). L'hameconnage l'emporte toujours."""
    # TODO
    pass

def niveau_risque(score_phishing, score_spam):
    """Faible (0-24), Moyen (25-49), Eleve (50-74), Critique (75-100)."""
    # TODO
    pass
```

À retenir

Ces seuils sont provisoires. Vous devez les **réajuster après la mesure des résultats**. Un seuil choisi au hasard puis jamais revu est une faute de méthode.

3.2 Sortie attendue

```
================================================
  SecuMail 2.0 - Analyse
================================================
Expediteur : Instagram Support <instagram.help2024@gmail.com>
Sujet : URGENT : ton compte INSTAGRAM sera SUPPRIME !!!

--- Analyse hameconnage ---
  +20 Nom affiche 'Instagram Support', domaine gmail.com non officiel
  +25 Le message parle de 'instagram' mais le lien mene vers
       instagram.com.securite-compte.info
  +25 Lien vers une adresse IP nue : 203.0.113.45
  +15 Reply-To detourne vers instagram-securite-compte.info
  Score : 100/100 (borne)

--- Analyse spam ---
  +15 Mots commerciaux : cadeau, gratuit, limitee, offre, selectionne, urgent
  +10 Formulation promotionnelle : 'Offre limitee'
   +5 6 points d'exclamation
  Score : 40/100

--- Decision ---
  Verdict : PHISHING
  Niveau de risque : Critique
  Action : Blocage + alerte analyste
================================================
```

3.3 Format des données

```
sender,name,subject,message,label
instagram.help2024@gmail.com,Instagram Support,Compte suspendu,Verifie ton
    compte,phishing
concours@concours-iphone-gratuit.xyz,Concours iPhone,GAGNE UN IPHONE,Clique
    vite,spam
marie.tremblay@monecole.qc.ca,Marie Tremblay,Sortie du 14 mars,Reportee a
    avril,ham
```

La colonne `label` est obligatoire : sans elle, aucune évaluation n'est possible.

4 Apprentissage machine

Jusqu'ici, vous avez expliqué à SecuMail comment reconnaître un courriel dangereux en écrivant vous-mêmes des règles. Vous avez choisi les indices à rechercher, les points à attribuer et les seuils à utiliser.

Dans cette nouvelle approche, nous ne donnerons pas directement ces règles au système. Nous lui montrerons plutôt plusieurs courriels dont nous connaissons déjà la bonne réponse. Le système tentera alors de découvrir les indices qui reviennent souvent dans chaque catégorie. La question à laquelle nous voulons répondre est la suivante :
Un système qui apprend à partir d'exemples est-il plus efficace que notre système fondé sur des règles ?

4.1 Les données utilisées

Chaque ligne du fichier représente un courriel. Elle contient notamment l'expéditeur, le nom affiché, l'objet, le message et la bonne réponse.

```
sender,name,subject,message,return_path,reply_to,label
direction@monecole.qc.ca,Direction,Rappel important,La sortie est reportee,,
,ham
promo@rabais-express.info,Super Rabais,OFFRE LIMITEE,Achetez maintenant,,
,spam
support@instagrarn-login.info,Instagram,Compte suspendu,Verifiez votre
    compte,,
,phishing
```

La colonne `label` contient la bonne réponse :
- `ham` : un courriel légitime ;
- `spam` : un message publicitaire ou indésirable ;
- `phishing` : une tentative d'hameçonnage.

Ces réponses sont appelées des **étiquettes**. Elles permettent au système de savoir ce qu'il doit apprendre.

À retenir

Le modèle apprend à partir des exemples qu'on lui donne. Si les étiquettes sont incorrectes ou si les exemples ne sont pas assez variés, il apprendra de mauvaises habitudes.

4.2 Entraînement et test final

Les données sont séparées en deux groupes qui ne contiennent pas les mêmes modèles de courriels :

1. les données d'**entraînement**, que le système peut observer pour apprendre ;

2. les données de **test final**, utilisées seulement pour vérifier son travail.

À retenir

Le modèle ne doit jamais apprendre avec les courriels du test final. Ce serait comme donner les questions d'un examen avant l'examen : une bonne note ne prouverait pas que l'élève sait résoudre de nouveaux problèmes.

4.3 Rassembler les informations du courriel

Le système fondé sur des règles examinait l'identité de l'expéditeur et le contenu du message. Pour que la comparaison soit juste, le modèle d'apprentissage automatique recevra les mêmes informations.

```
def construire_texte(data):
  return (
      data["name"].fillna("") + " " +
      data["sender"].fillna("") + " " +
      data["return_path"].fillna("") + " " +
      data["reply_to"].fillna("") + " " +
      data["subject"].fillna("") + " " +
      data["message"].fillna("")
  )

X_entrainement = construire_texte(entrainement)
y_entrainement = entrainement["label"]

X_test = construire_texte(test_final)
y_test = test_final["label"]
```

La variable `X` contient les informations que le système peut observer. La variable `y` contient la bonne réponse attendue.

4.4 Transformer les mots en nombres

Un programme ne comprend pas directement les phrases comme un humain. La classe instanciée `TfidfVectorizer` transforme donc les mots et les groupes de mots en nombres.

Un mot qui apparaît dans presque tous les courriels, comme « bonjour », apporte généralement peu d'information. Un mot ou une expression qui revient surtout dans une catégorie peut être plus utile pour prendre une décision.

Le modèle pourrait par exemple remarquer que les expressions suivantes apparaissent souvent dans certains courriels :

```
"offre limitee"
"compte suspendu"
"cliquez maintenant"
"reunion de demain"
"monecole.qc.ca"
```

Cela ne signifie pas qu'il comprend réellement ces expressions. Il observe simplement les associations présentes dans ses exemples.

4.5 Construire et entraîner le modèle

Nous utiliserons un seul modèle composé de deux étapes :

1. `TfidfVectorizer` transforme le texte en nombres ;
2. `LogisticRegression` utilise ces nombres pour choisir entre `ham`, `spam` et `phishing`.

Malgré son nom, `LogisticRegression` peut servir à classer des éléments dans plusieurs catégories.

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

modele = Pipeline([
  ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
  ("classificateur", LogisticRegression(max_iter=1000))
])

modele.fit(X_entrainement, y_entrainement)
```

La commande `fit()` représente l'entraînement. Le modèle observe les courriels et leurs étiquettes afin de découvrir les associations qui peuvent l'aider à prendre une décision.

4.6 Faire des prédictions

Une fois entraîné, le modèle peut analyser les courriels du test final.

```
predictions = modele.predict(X_test)

for attendu, obtenu in zip(y_test[:10], predictions[:10]):
  print("Attendu :", attendu, "| Prediction :", obtenu)
```

Une prédiction peut être correcte ou incorrecte. Le modèle peut notamment :

— reconnaître correctement un hameçonnage ;

— confondre un spam avec un hameçonnage ;
— bloquer un courriel légitime ;
— laisser passer un courriel dangereux.

À retenir

Une prédiction n'est pas une certitude. Même si le modèle a obtenu de bons résultats pendant les tests, il peut toujours se tromper sur un nouveau courriel.

4.7 Mesurer les résultats

Nous utiliserons une **matrice de confusion** pour compter les bonnes réponses et les différents types d'erreurs.

```
from sklearn.metrics import classification_report
from sklearn.metrics import ConfusionMatrixDisplay

print(classification_report(
  y_test,
  predictions,
  zero_division=0
))

ConfusionMatrixDisplay.from_predictions(
  y_test,
  predictions
)
```

Trois mesures sont particulièrement utiles :
— la **précision** : lorsque le système signale une catégorie, a-t-il généralement raison ?
— le **rappel** : parmi les vrais courriels de cette catégorie, combien ont été trouvés ?
— le score **F1** : un résumé qui tient compte de la précision et du rappel.

À retenir

L'exactitude ne doit pas être utilisée seule. Si 96 % des courriels sont légitimes, un système qui répond toujours `ham` obtient 96 % d'exactitude, même s'il ne détecte aucune attaque.

4.8 Comprendre les erreurs

Choisissez quelques erreurs commises par le modèle et examinez les courriels concernés. Pour chaque erreur, répondez aux questions suivantes :

1. Quelle était la bonne réponse ?
2. Quelle réponse le modèle a-t-il donnée ?
3. Quels mots, adresses ou expressions ont pu le tromper ?

4. Le système fondé sur des règles aurait-il fait la même erreur ?
5. Quelle serait la conséquence de cette erreur dans une vraie messagerie ?

4.9 Mettre le modèle au défi

Un attaquant peut modifier son message pour tenter de contourner un système de sécurité. Testez votre modèle avec les situations suivantes :

1. un hameçonnage écrit calmement, sans utiliser les mots « urgent » ou « compte suspendu » ;
2. un courriel scolaire légitime contenant « GRATUIT » ou plusieurs points d'exclamation ;
3. un faux domaine appartenant à une marque absente des données d'entraînement ;
4. un spam dont les mots les plus évidents ont été remplacés par des synonymes.

Notez la prédiction obtenue pour chaque essai. Comparez ensuite le résultat avec celui du système fondé sur des règles.

4.10 Comparer les deux approches

Les deux systèmes doivent être évalués sur les **mêmes courriels du test final**.

Approche	Précision phishing	Rappel phishing	F1 macro	Courriels légitimes bloqués
Système fondé sur des règles				
Apprentissage automatique				

Les résultats chiffrés ne sont pas les seuls éléments à comparer :

Question	Système fondé sur des règles	Apprentissage automatique
Peut-on expliquer facilement une décision ?		
Faut-il écrire les indices à la main ?		
Faut-il posséder beaucoup d'exemples ?		
Le système reconnaît-il de nouvelles formulations ?		
Est-il facile pour un attaquant de le tromper ?		

5 Tâches optionnelles – pour aller plus loin

Ces trois prolongements ne sont pas exigés, mais ils sont fortement valorisés dans le rapport. Les deux premiers exigent des fichiers `.eml` complets : un CSV contenant seulement `sender`, `subject` et `message` ne contient pas les en-têtes.

5.1 Authentification de l'expéditeur : SPF, DKIM et DMARC

Jusqu'ici, vous avez comparé du *texte*. Ces trois mécanismes font autre chose : ils vérifient la **cryptographie et le DNS**. C'est la seule partie du courriel qu'un attaquant ne peut pas simplement récrire.

Mécanisme	Question posée	Comment
SPF	Ce serveur figure-t-il dans la liste des serveurs autorisés du domaine ?	Le domaine publie ses IP autorisées dans un enregistrement DNS `TXT`.
DKIM	Le message a-t-il été modifié depuis son envoi ?	L'expéditeur signe le message ; la clé publique est dans le DNS.
DMARC	Le domaine du `From` *affiché* est-il aligné avec celui vérifié par SPF ou DKIM ?	Le seul des trois qui protège la ligne que l'utilisateur voit ; il dicte l'action : `none`, `quarantine` ou `reject`.

À retenir

Trois erreurs de raisonnement à ne pas commettre :

1. *SPF absent = fraude.* Faux : beaucoup de petits domaines n'ont rien publié. Absence n'est pas preuve, la règle doit renvoyer `(0, "")`.
2. *SPF pass = courriel sûr.* Faux : un attaquant qui possède `desjardins-verification.info` peut parfaitement le configurer avec un SPF et un DKIM valides. C'est l'**alignement DMARC** qui donne son sens à l'ensemble.
3. *Je vais vérifier SPF moi-même en code.* Non : la vérification a lieu *à la réception*, par le serveur, qui connaît l'IP de connexion. Vous, vous **lisez son verdict** dans l'en-tête `Authentication-Results`.

```
import email
from email import policy

MOTIF_AUTH = re.compile(
    r"\b(spf|dkim|dmarc)\s*=\s*"
    r"(pass|fail|softfail|neutral|none|permerror|temperror)", re.I)

def lire_eml(chemin):
    """Ouvre un .eml. On ne l'execute pas, on ne l'affiche pas : on le LIT."""
    # TODO
    pass

def resultats_authentification(message):
    """Lit Authentication-Results et Received-SPF.
       -> {"spf": "pass", "dkim": "fail", "dmarc": "fail"}"""
    # TODO
    pass

def regle_authentification(resultats, mecanisme, points=20):
    """+20 si le mecanisme a echoue. Absent des resultats -> (0, "")."""
    # TODO
    pass
```

À traiter dans le rapport. Un courriel obtient `spf=pass`, `dkim=pass`, `dmarc=fail`. Que s'est-il passé ? Quel est le domaine réellement authentifié ? Quel verdict rendez-vous, et pourquoi vos règles du module 1 restent-elles indispensables ?

5.2 Analyse des pièces jointes et interrogation de VirusTotal

À retenir

Cadre obligatoire – non négociable.

- **N'ouvrez jamais** une pièce jointe analysée. Vous la lisez en binaire, vous calculez son empreinte : vous ne l'exécutez pas.
- **N'envoyez jamais le fichier** à VirusTotal, uniquement son empreinte **SHA-256**. Téléverser un fichier le rend public : un document d'entreprise déposé sur VirusTotal est une fuite de renseignements, susceptible de contrevenir à la **Loi 25**.
- **La clé d'API ne figure jamais dans le code**, ni dans un dépôt Git : elle se lit dans une variable d'environnement.
- Travaillez uniquement sur des échantillons de laboratoire fournis par l'enseignant. L'accès non autorisé à un système informatique est une infraction (*art.* 342.1 du *Code criminel*).

```
import hashlib, os, requests

EXTENSIONS_DANGEREUSES = {".exe", ".scr", ".js", ".vbs", ".jar", ".bat",
                          ".cmd", ".ps1", ".hta", ".iso", ".lnk", ".msi"}
EXTENSIONS_MACRO = {".docm", ".xlsm", ".pptm", ".xlsb"}
EXTENSIONS_LEURRE = {".pdf", ".doc", ".docx", ".xls", ".xlsx", ".jpg", ".txt"}

def pieces_jointes(message):
    """Liste de couples (nom_de_fichier, contenu_binaire)."""
    # TODO
    pass

def empreinte_sha256(contenu_binaire):
    """Identifie le fichier sans jamais le divulguer."""
    # TODO
    pass

def regle_extension_piece_jointe(nom_fichier):
    """+35 executable ; +30 double extension (facture.pdf.exe) ;
       +15 document a macros ; sinon (0, ")."""
    # TODO
    pass

def interroger_virustotal(sha256, cle_api=None):
    """GET https://www.virustotal.com/api/v3/files/<sha256>, entete x-apikey.
       -> {"connu": bool, "malveillants": int, "moteurs": int} ou None."""
    # TODO
    pass

def regle_virustotal(rapport, seuil=3, points=40):
    """+40 si au moins ‘seuil‘ moteurs classent le fichier malveillant."""
    # TODO
    pass
```

Deux limites à écrire dans le rapport. Un fichier *inconnu* de VirusTotal n'est pas inoffensif : un maliciel fabriqué le matin même n'y figure pas, et c'est précisément le cas des attaques ciblées. Le compte gratuit est par ailleurs limité à environ 4 requêtes par minute et 500 par jour : un traitement en lot doit être temporisé et mettre en cache les empreintes déjà vues.

5.3 Analyse avancée des liens

Au module 1, le lien servait à démasquer une *usurpation d'identité*. Ici, il sert à caractériser une *campagne commerciale* : combien de liens, vers combien de domaines, avec quelle proportion de texte et surtout, le texte du lien dit-il la vérité sur sa destination ?

```
MOTIF_ANCRE = re.compile(
    r"<a[^>]+href=[\"'](?P<href>[^\"']+)[\"'][^>]*>(?P<texte>.*?)</a>",
    re.I | re.S)
MOTIF_DOMAINE_TEXTE =
    re.compile(r"\b([a-z0-9-]+\.[a-z]{2,})(?:\.[a-z]{2,})?\b", re.I)
MOTS_DESABONNEMENT = ["desabonner", "desabonnement", "unsubscribe"]

def regle_ancre_trompeuse(html):
    """+25 (HAMECONNAGE) si le texte visible d'un lien annonce un domaine
       different de celui reellement vise par le href."""
    # TODO
    pass

def regle_domaines_distincts(texte, seuil=3):
    """+10 si le message pointe vers au moins `seuil` domaines DIFFERENTS."""
    # TODO
    pass

def regle_densite_liens(texte, seuil=0.10, mots_min=20):
    """+10 si nb_liens / nb_mots depasse `seuil`.
       En dessous de `mots_min` mots : (0, "")."""
    # TODO
    pass

def regle_desabonnement(texte):
    """-5 : presence d'un lien de desabonnement (indice de LEGITIMITE)."""
    # TODO
    pass
```

À retenir

`regle_desabonnement` est votre **première règle à points négatifs** : la *Loi canadienne anti-pourriel* (LCAP) impose un mécanisme de désabonnement fonctionnel à tout envoi commercial. Un vrai commerçant s'y conforme, un escroc rarement. Reconnaître un indice *en faveur* du message est ce qui distingue un système d'analyse d'un simple détecteur d'alarme, mais cela impose de **borner le score à 0** dans le moteur, sinon un message peut finir avec un score négatif et casser vos seuils.

6 Récapitulatif des poids

Règle	Points	Module
`regle_mot_appat`	+25	hameçonnage
`regle_extension_risque`	+10	hameçonnage
`regle_substitution`	+30	hameçonnage
`regle_lien_incoherent`	+25	hameçonnage
`regle_lien_adresse_ip`	+25	hameçonnage
`regle_lien_raccourcisseur`	+15	hameçonnage
`regle_lien_sans_https`	+10	hameçonnage
`regle_nom_vs_adresse`	+20	hameçonnage
`regle_return_path`	+15	hameçonnage
`regle_reply_to`	+15	hameçonnage
`regle_authentification` (×3, optionnel)	+20 chacune	hameçonnage
`regle_extension_piece_jointe` (optionnel)	+15 à +35	hameçonnage
`regle_virustotal` (optionnel)	+40	hameçonnage
`regle_ancre_trompeuse` (optionnel)	+25	hameçonnage
`regle_mots_commerciaux`	+3/mot, max 15	spam
`regle_majuscules`	+10	spam
`regle_formulation_promo`	+10	spam
`regle_exclamations`	+5	spam
`regle_nombre_liens`	+10	spam
`regle_domaines_distincts` (optionnel)	+10	spam
`regle_densite_liens` (optionnel)	+10	spam
`regle_desabonnement` (optionnel)	**−5**	spam

7 Prérequis Python

Niveau	Contenu	Nécessaire pour
Base	variables, listes, dictionnaires, boucles, conditions, fonctions, `try/except`	tout le projet
Chaînes et regex	`.lower()`, `.split()`, `.endswith()`, `.count()`, module `re`	modules 1 et 2
Fichiers	lecture CSV (`csv` ou `pandas`)	données
pandas	`read_csv`, filtrage, `value_counts()`	séance 4
scikit-learn	`fit` / `predict`, `train_test_split`, `TfidfVectorizer`, `classification_report`	séances 4 et 5

Environnement recommandé : **Google Colab**. Aucune installation, les bibliothèques sont déjà présentes, et le partage se fait par lien.

8 Cadre éthique et légal

— **Aucun courriel réel** d'un camarade, d'un proche ou d'un employeur sans autorisation écrite.
— **Aucun envoi** des courriels de test vers une messagerie active : de vraies personnes recevraient les rebonds.
— **Aucune ouverture** de pièce jointe suspecte, aucune visite des liens analysés.
— L'accès non autorisé à un système informatique est une infraction (*art.* 342.1 du *Code criminel*).

9 Ressources

Documentation

— `re` – expressions régulières : docs.python.org
— scikit-learn – guide de démarrage : scikit-learn.org
— Comprendre SPF, DKIM et DMARC : documentation de votre fournisseur de messagerie